Phase	Phase Name	Main Thing It Does
0	**Vision Lock**	Defines exactly what Obsidian Protocol is, what it must be, and what it must never become.
1	**Unreal Project Foundation**	Establishes the Unreal project, source structure, conventions, builds, version control, and baseline.
2	**Engine & Plugin Audit**	Determines and locks the Unreal plugins and engine capabilities the game actually needs.
3	**Core Game Architecture**	Builds the fundamental game framework: GameInstance, GameMode, GameState, players, authority, replication, persistence, and events.
4	**Data Architecture**	Defines how units, weapons, resources, missions, intents, AI, damage, visuals, audio, and other game data are represented.
5	**Core Unit Framework**	Creates the universal unit foundation that every military unit builds upon.
6	**Reserved / Not in Supplied Master Checklist**	No Phase 06 was defined in the master checklist you supplied.
7	**Ground Unit Framework**	Makes ground vehicles physically move, navigate terrain, take damage, recover, communicate, and operate autonomously.
8	**Sea Unit Framework**	Makes naval units operate in water with buoyancy, waves, depth, navigation, sensors, damage, and autonomy.
9	**Command Units**	Creates the command hierarchy and command network that coordinates the military.
10	**Experimental Units**	Provides the framework for unusual/restricted units and capabilities without breaking the core architecture.
11	**Sensor & Detection System**	Determines what units can actually see, detect, identify, track, and know about the battlefield.
12	**Communication & Information Warfare**	Controls how information moves, degrades, becomes stale, gets blocked, or is disrupted.
13	**Command Intent System**	Gives the player the ability to tell forces what they need to accomplish and under what rules.
14	**Autonomous AI**	Turns player intent and battlefield information into autonomous decisions, actions, adaptation, and coordination.
15	**Navigation**	Makes units realistically navigate ground, air, sea, formations, obstacles, and changing routes.
16	**Combat Foundation**	Establishes the fundamental combat mechanics: targeting, weapons, projectiles, hitscan, damage, armor, penetration, destruction, and feedback.
17	**Advanced Combat**	Expands combat into combined-arms warfare, air/ground/naval combat, suppression, support, targeting, retreats, resource management, and tactical consequences.
18	**Deployment System**	Controls how players assemble and deploy forces while enforcing the fixed Deployment Budget.
19	**Resource System**	Creates the eight-resource economy and makes resources usable, authoritative, transferable, persistent, and meaningful.
20	**Manufacturing / Fabrication**	Creates production, fabrication, repair, salvage, recycling, queues, materials, and manufacturing requirements.
21	**Player Ownership / Fleet**	Creates the player's persistent military inventory, fleet organization, storage, condition, customization, and history.
22	**Garage / Facility**	Builds the player's physical military headquarters where they manage and interact with their persistent military.
23	**Player Interaction**	Makes the facility, vehicles, equipment, command systems, and other objects physically interactable.
24	**VR**	Makes the entire military command experience work as a full VR experience using the same core systems as PC.
25	**World System**	Builds the actual battlefield world: terrain, roads, structures, vegetation, water, boundaries, cover, elevation, weather, and lighting foundations.
26	**Environmental Simulation**	Makes weather, wind, visibility, temperature, terrain, water, and day/night affect gameplay systems.
27	**Destruction / Damage World**	Makes the physical battlefield change through destruction, debris, altered collision/navigation, and persistent damage.
28	**Multiplayer Foundation**	Makes the actual multiplayer architecture work: servers, clients, replication, authentication, joining, reconnecting, validation, and persistence.
29	**Scale / Massive Battle Testing**	Determines how many units the game can support and identifies/solves CPU, GPU, AI, physics, navigation, networking, and memory bottlenecks.
30	**UI Foundation**	Builds the underlying UI architecture that all menus, windows, panels, popups, navigation, input, and overlays use.
31	**RTS HUD**	Creates the main battlefield command interface and displays resources, units, sensors, objectives, autonomy, alerts, and commands.
32	**Autonomy / Intent UI**	Gives the player visual tools for creating and managing Command Intent, priorities, ROE, behaviors, and autonomy.
33	**Deployment UI**	Creates the interface for building an army and managing deployment zones, eligibility, and the Deployment Budget.
34	**Command / Strategic UI**	Creates strategic and battlefield maps, intelligence, command hierarchy, logistics, communications, objectives, and threats.
35	**Audio**	Creates the complete reactive audio identity: music, weapons, vehicles, sensors, communications, ambience, UI, VR, and mixing.
36	**Visual Identity**	Establishes the recognizable Obsidian Protocol visual language across units, facilities, vehicles, UI, materials, lighting, effects, and factions.
37	**Unit Content Production**	Produces every actual military unit to final production standards across models, data, audio, AI, combat, damage, multiplayer, and performance.
38	**Facility Content Production**	Produces every physical facility area to final production standards, including architecture, props, lighting, audio, interaction, VR, and performance.
39	**Map Production**	Produces complete playable battlefields with terrain, objectives, resources, deployment zones, navigation, AI, multiplayer, art, audio, and QA.
40	**Progression**	Builds the player's long-term advancement, unlocking, research, manufacturing progression, garage progression, and multiplayer progression.
41	**Monetization**	Builds the credits/store system while enforcing the game's anti-pay-to-win and no-deployment-bypass rules.
42	**Save / Persistence**	Makes player, fleet, resources, progression, customization, garage, campaign, and multiplayer data persist safely.
43	**Security / Anti-Cheat**	Protects authoritative systems against exploits, manipulated resources, movement, damage, inventory, purchases, progression, and network attacks.
44	**Accessibility**	Makes the game usable across different accessibility needs involving UI, controls, audio, color, motion, subtitles, and VR comfort.
45	**Performance Optimization**	Optimizes the entire game for CPU, GPU, AI, networking, physics, rendering, memory, streaming, and large battles.
46	**QA Foundation**	Establishes the formal testing, bug reporting, regression, automated testing, multiplayer, AI, performance, save, and VR testing systems.
47	**Full System Integration**	Connects the entire game into one continuous loop from entering the game → preparing the military → deploying → fighting → results → persistence → returning to the facility.
48	**Internal Alpha**	Produces the first genuinely playable internal version of the complete game and stabilizes the major systems.
49	**Closed Alpha**	Gives external testers the game so real players can expose problems with the core experience, AI, UI, multiplayer, VR, performance, and UX.
50	**Beta**	Brings the game to feature/content complete status and performs broad balance, compatibility, networking, persistence, security, and optimization work.
51	**Balance**	Tunes units, weapons, resources, deployment, AI, maps, progression, economy, match length, and strategic diversity.
52	**Final Art Pass**	Replaces every remaining placeholder and brings units, environments, facilities, UI, animation, VFX, materials, textures, audio, cinematics, and marketing visuals to final quality.
53	**UX / Player Experience Pass**	Plays the game as a completely new player and verifies that the entire experience communicates the game's unique identity and gameplay loop.

54	**Final Multiplayer Test**	Performs the final multiplayer stress and reliability testing across player counts, latency, packet loss, disconnects, recovery, servers, and exploits.
55	**Platform Certification**	Prepares and validates the game against the technical, account, controller, network, privacy, store, performance, and certification requirements of each target platform.
56	**Release Build**	Creates the actual production client/server build and verifies all backend, authentication, matchmaking, persistence, commerce, support, security, and recovery systems.
57	**Final Release Candidate**	Freezes feature development and allows only critical fixes while performing final regression, performance, multiplayer, security, platform, store, art, audio, UI, and localization checks.
58	**Launch Preparation**	Prepares everything surrounding release: store page, trailer, screenshots, documentation, community, support, marketing, server capacity, monitoring, and incident response.
59	**Release**	Ships Obsidian Protocol: production build, servers, authentication, matchmaking, persistence, commerce, monitoring, and support go live.
60	**Launch Day**	Actively monitors the live launch and handles crashes, servers, networking, economy, purchases, exploits, AI failures, performance, rollback, hotfixes, and recovery.
61	**Post-Launch**	Analyzes the first day, week, and month to understand crashes, performance, player behavior, economy, AI, balance, community feedback, and future improvements.
62	**Live Game**	Continues operating and expanding the game through patches, security, balance, units, maps, missions, facility content, cosmetics, events, scaling, and justified new systems.